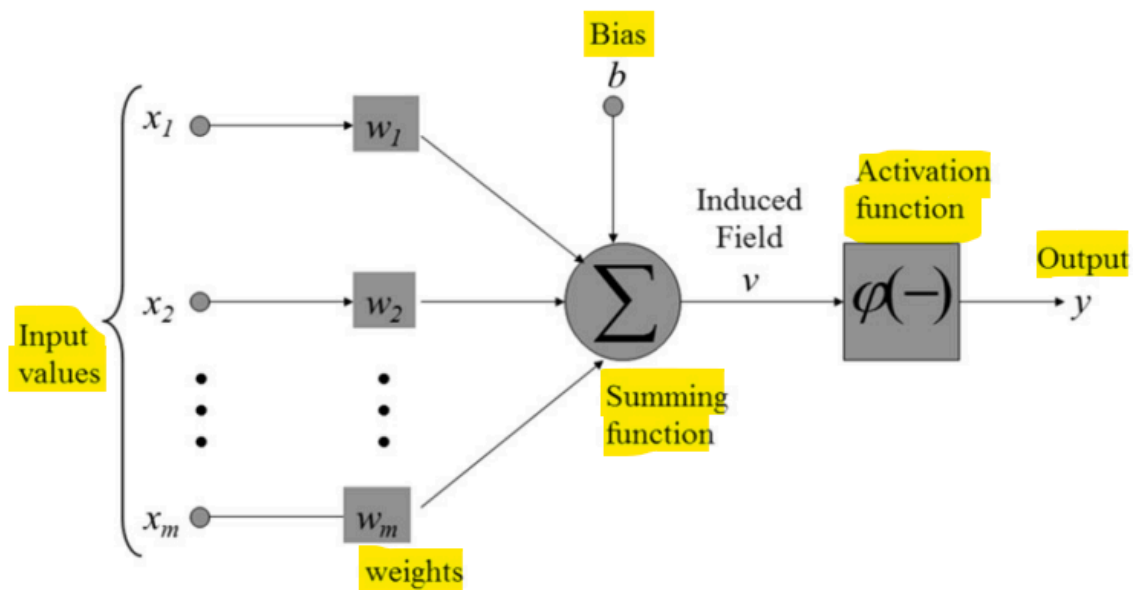
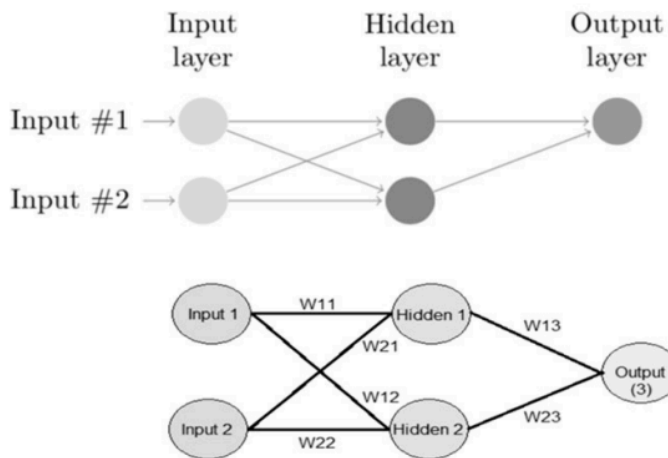


Neural Information Retrieval

(Artificial) Neural Network [ANN]

- Neural networks is inspired by the **human brain**
- NN is a **collection of many artificial neurons connected together.**
- It **learns patterns** from data.





★ Inputs ($x_1, x_2, \dots, x_m$):

- These are the **values/features** given to the neuron.

★ Weights ($w_1, w_2, \dots, w_m$)

- **Each input has a weight attached** to it.
- Weights decide:
 - ➡ **How important an input is.**
 - Large weight = more influence
 - Small weight = less influence
 - Negative weight = opposite effect

★ Bias (b)

- Bias is an **extra value added to the neuron.**
- It helps the neuron:
 - ✓ Shift the output
 - ✓ **Learn patterns better**
 - ✓ **Improve flexibility**

Think of it like an **“adjustment value”**.

★ Summation Function (Σ)

- neuron **multiplies inputs with weights** and **adds them**

$$v = \sum_{i=1}^m w_i x_i + b$$

- $v \Rightarrow$ **induced field**.

★ Activation Function (ϕ)

- Decides:  **Should the neuron activate or not?**
- It **converts** the **summed value into** the **final output**.
- Common activation functions:
 - **Sigmoid**
 - **ReLU**
 - **Tanh**
- Without activation functions, neural networks cannot learn complex patterns.

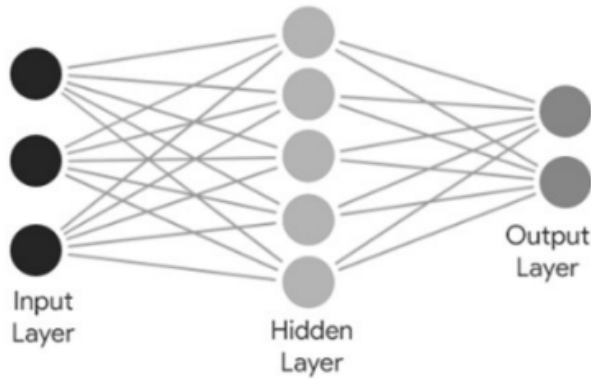
★ Output (y)

- **Final result produced** by neuron
-

Working of an Artificial Neuron

Step-by-step:

1. Receive inputs
2. Multiply each input by its weight
3. Add all values + bias
4. Pass result through activation function
5. Produce output



1. Input Layer

- This layer receives data from the outside world.
- Example:
 - Input #1
 - Input #2
- No processing happens here.
It **only passes data forward**.

2. Hidden Layer

- This is where learning happens.
- The hidden neurons:
 - ✓ **Process inputs**
 - ✓ **Detect patterns**
 - ✓ **Perform calculations**
- A network can have:
 - One hidden layer
 - **Multiple hidden layers**

More hidden layers = Deep Learning



3. Output Layer

- Produces the **final prediction/result**.
- Examples:
 - 0 or 1
 - Disease detected
 - Class label

> the lines between neurons represent connections

> each connection has a weight!

- $W_{11} \Rightarrow$ input 1 to layer 1
 - $W_{12} \Rightarrow$ input 1 to layer 2
 - $W_{21} \Rightarrow$ input 2 to layer 1
-

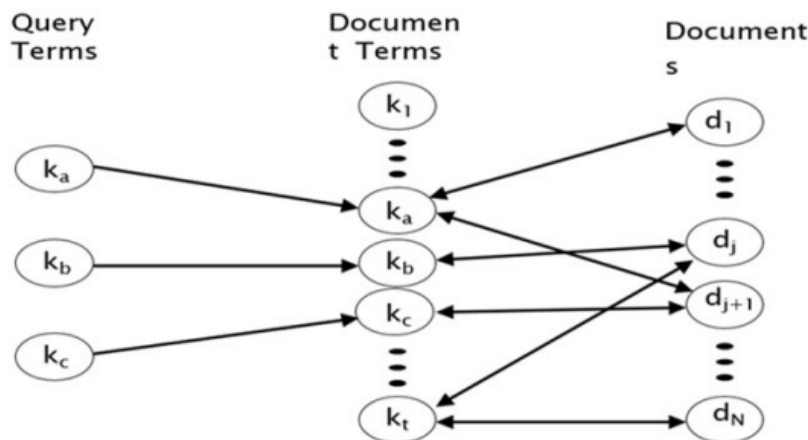


Neural Information Retrieval

Neural Information Retrieval is an early neural-network-based approach for:

- ✓ **Searching documents**
- ✓ **Finding relevant information**
- ✓ **Ranking documents according to a query**

- This model was proposed by: 📖 **Wilkinson & Hingston (SIGIR 1991)**



- The system uses a **3-layer neural network**.
- When a user enters a query:

- **Signals start from query terms**
- Signals travel through the network
- **Relevant documents get activated**

- The **higher the activation**:
 ✓ The **more relevant the document** is.



1. Query Term Layer

- This layer **contains the query words** entered by the user.
- **Each query term sends an initial signal into the network.**
- Example nodes:

- k_a
- k_b
- k_c

- These are **query-term neurons**.



2. Document Term Layer

- This **middle layer** contains **terms/features from documents**.
- It acts like a **bridge between**:
 - **Query terms**
 - **Documents**
- Purpose:
 - ✓ **Match query words with document words**
 - ✓ **Spread activation signals**



3. Document Layer

This layer **contains** the **actual documents**:

- d_1
- d_j
- d_N

Documents receiving **strong signals** are **considered** ★ **More relevant** to the query.



Feedback Signals

The slide also says: “**Document nodes might themselves generate new signals**”

Documents can send signals back to term nodes.

This creates:

- ✓ **Reinforcement**
- ✓ **Better matching**
- ✓ **Spreading activation**

“After the signals passed to the two layers the **output layers neurons** – get **activated by the dot product of document and query vector terms**”

- The output layer computes similarity using: **Dot Product / Cosine Similarity**
- Both **query** and **documents** are represented as **vectors**.
- The similarity formula used is:

$$\text{Similarity}(Q, D) = \frac{\sum_i w_{iq} w_{ij}}{\sqrt{\sum_i w_{iq}^2} \sqrt{\sum_i w_{ij}^2}}$$

Formula measures: **how similar the doc is to the query**

- w_{iq} = weight of term i in the query
- w_{ij} = weight of term i in document j

Higher value = more relevance

Final Ranking

The system:

- ✓ **Collects all document nodes**
- ✓ **Calculates activation/similarity scores**
- ✓ **Sorts documents in decreasing order**

So:

Highest score → **Most relevant** document

Lowest score → **Least relevant** document

Lexical Semantics

Lexical Semantics is the study of:

- ✓ **Word meanings**
- ✓ **Relationships between words**
- ✓ How **words combine to create meaning**

It focuses on the **meaning of words in a language system** (dictionary-style meaning).

It also studies:

- **Synonyms**
 - **Antonyms**
 - **Multiple meanings** of words [**Polysemy**]
-
- ❑ **Synonymy**: "Quick" and "Fast" are synonyms.
 - ❑ **Antonymy**: "Big" and "Small" are antonyms.
 - ❑ **Polysemy**: "Light" can mean both a source of illumination and something not heavy.
-

Vector Semantics

Vector Semantics represents words as:

- **Mathematical vectors** (numbers)
- **Points in high-dimensional space**

Instead of fixed dictionary meanings, **meaning is learned from data**.

Main Idea of Vector Semantics

Based on the: **Distributional Hypothesis**

"Words appearing in similar contexts have similar meanings."

Meaning:

If two words occur with similar surrounding words, they probably have similar meanings.

How It Works

Meaning of a word is derived in the following way:

It checks:

- ✓ **Which words appear together**
- ✓ **How often they co-occur**

Then it **creates vector representations**.

Word Embeddings

A word embedding is:

→ A **dense vector representation of a word**

Popular embedding methods:

- Word2Vec
- GloVe
- FastText

✨ Famous Example

Vector arithmetic: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$

This works because **vector relationships capture semantic meaning**.

🧠 Why This is Amazing

The model learns:

- **Gender relationships**
- **Similarity**
- **Contextual meaning**

without explicitly programming rules.

Lexical Semantics vs Vector Semantics

Aspect	Lexical Semantics	Vector Semantics
Representation	Discrete, symbolic representation of word meanings	Continuous, dense vector representation of words <i>numeric</i>
Meaning Source	Defined by linguistic relationships (synonyms, etc.)	Derived from the distribution of words in a large corpus
Word Relations	Explicit (e.g., synonymy, antonymy)	Implicit (similarity measured via distance in vector space)
Context Sensitivity	Context-independent (meaning is fixed)	Context-sensitive (meaning <u>depends</u> on <u>word co-occurrence</u>)
Example of Word Meaning	"Dog" means a type of domesticated animal	"Dog" is close to "cat" and "wolf" in vector space
Scope	Focuses on relationships within a predefined lexicon	Focuses on relationships between words learned from data

Words as Vectors

In Vector Semantics, words are represented as:

- **Vectors (lists of numbers)**
- Points in high-dimensional space

Instead of storing words as plain text,
the computer converts them into mathematical representations.

Each word gets:

- ✓ **A fixed-length vector**
- ✓ Made of **real numbers**
- ✓ Representing the word's meaning

Example:

Word	Vector Representation
King	[0.21, -0.44, 0.91, ...]
Queen	[0.25, -0.40, 0.95, ...]

These numbers capture semantic meaning.

- **Each word** in vocab is **mapped to a point in the vector space**
 - Words with **similar meanings appear near each other** in vector space.
-



Dense Vector Representation

The slides mention: **"Each word is represented by a dense vector with n dimensions"** (**EMBEDDING**)

- $n \ll \text{vocab size}$

Dense means: **Most vector values contain meaningful numbers.**

Example: Almost all positions have useful values.

Dense Vector

[0.42, -0.11, 0.87, 0.63]

Before embeddings, words were represented using:

◆ One-Hot Encoding

Example vocabulary:

[cat, dog, fish]

Representations

Word	Vector
Cat	[1,0,0]
Dog	[0,1,0]
Fish	[0,0,1]

⚠ Problem with One-Hot Encoding

- ✗ Sparse
- ✗ Huge for large vocabularies
- ✗ Cannot capture similarity

✨ Word Embeddings

- Word embeddings solve problems w one hot encoding

Word embeddings are:

- ✓ **Dense, continuous vector representations** of words
- ✓ **Learned automatically** from text data
- ✓ **Semantically meaningful**

Words with **similar meanings get similar vectors** + are **placed closer together**

Embeddings help machines:

- ✓ **Understand meaning**
- ✓ **Detect similarity**
- ✓ **Learn context**
- ✓ **Improve NLP tasks**



How Embeddings are Learned

Embeddings are trained using:

- **Huge text datasets** (corpora)
- **Word co-occurrence patterns**

The system learns: “Which words appear together?”



Popular Embedding Models

- 1) **Word2Vec** → learns embeddings using **neighbouring words**
 - 2) **GloVe** → uses **global word co-occurrence statistics**
 - 3) **FastText** → represents words using **smaller subword units**
- They allow **ML models** to **understand relationships** between words **based on** their **meaning** and **contexts**
-



Embedding Dimensions

Typical embedding sizes:

- 50 dimensions
- 100 dimensions
- 200 dimensions
- 300 dimensions

Higher dimensions:



Capture more information



Need more computation



One-Hot vs Word Embeddings

Feature	One-Hot Encoding	Word Embeddings
Representation	Sparse	Dense
Captures Meaning	✗ No	✓ Yes
Captures Similarity	✗ No	✓ Yes
Size	Very Large	Compact
Learns Context	✗ No	✓ Yes



Word Embedding?

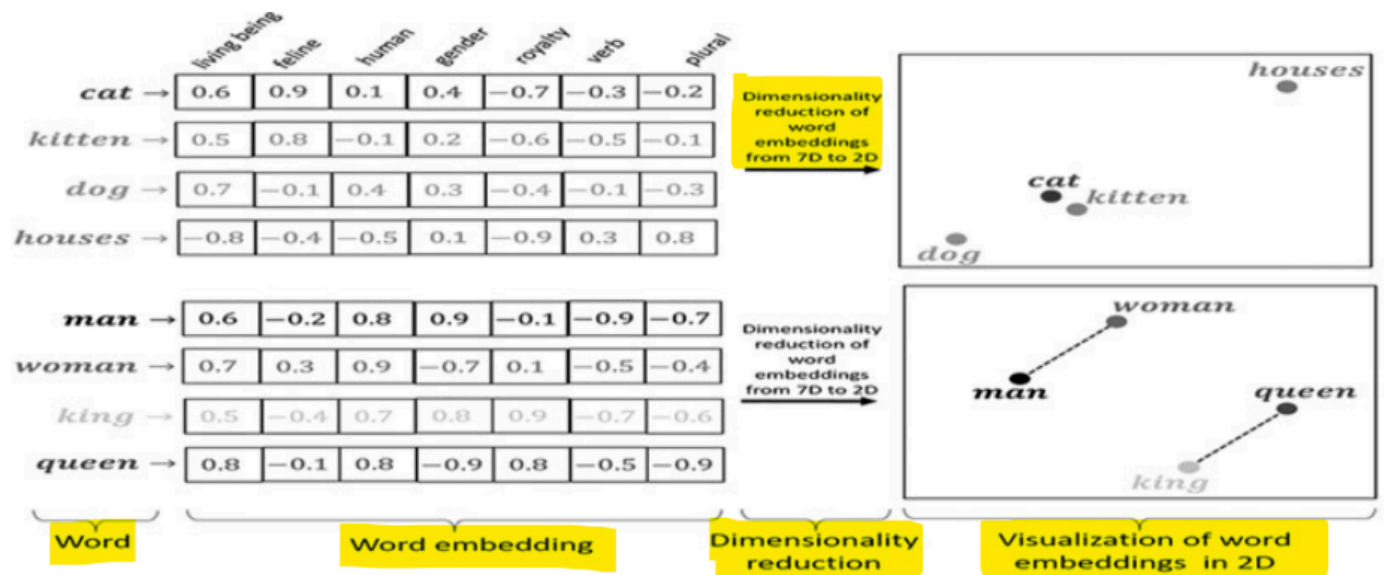
Word Embedding is a **technique in NLP** where:

- Words are converted into vectors (numbers)
- Similar words are placed close together in vector space

Instead of treating words as simple symbols, the **computer learns their meanings mathematically**.

Word	Vector
Cat	[0.6, 0.9, 0.1, ...]
Dog	[0.7, -0.1, 0.4, ...]

Diagram Explanation



◆ Left Side → Word Embedding Table

This table shows:

- ✓ Each word converted into numbers
- ✓ Every **column** = **feature/dimension**

Meaning of Columns

The **columns** represent **learned semantic features**.

Examples: living being, feline, human, etc

Similar Words Have Similar Vectors

Notice: **Cat and kitten** have very **similar values**.

So the model learns: ✓ They are **semantically related**.

◆ **Right Side → 2D Visualization**

The original embeddings may have:

- 70 dimensions
- 100 dimensions
- 300 dimensions

But **humans cannot visualize high dimensions**.

So:

- **Dimensionality reduction** is applied
- **Embeddings** are **projected into 2D**

🧠 What It Shows

✅ **Cat and kitten are very close**
because their meanings are **similar**.

🐶 **Dog is nearby**
because it is also an animal.

🏠 **Houses is far away**
because it belongs to a different category.



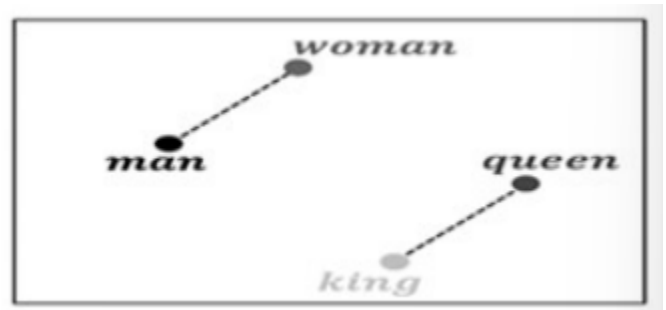
The relationships are preserved mathematically.

Example:

- king → queen
- man → woman

The vectors capture:

- ✅ **Gender relationships**
- ✅ **Semantic similarity**



? Why Word Embeddings?

♦ 1. Better Understanding of Language [linguistic analysis]

Embeddings help computers understand:

- ✅ **Meaning**

✓ **Context**

✓ **Similarity between words**

Better than older NLP methods.

◆ **2. Dimensionality Reduction**

Older methods produced: ✗ **Huge sparse matrices**

Embeddings use:

✓ **Smaller dense vectors**

This **reduces memory** and **complexity**.

◆ **3. Faster Training**

Because embeddings are **compact**:

✓ **Less computation** needed

✓ **Faster model training**

◆ **4. Dense Representation**

Embeddings are: ✓ **Dense vectors**

instead of sparse vectors.

Dense vectors:

- **Store more meaningful information**
 - **Are computationally efficient**
-



Types of Word Embeddings

There are 3 major types.

◆ 1. Count-Based Embeddings

Based on: ➡ **Word co-occurrence counts**

Idea: **Words appearing together frequently are related.**



Co-occurrence Matrix

A table storing: **How often words appear together**



Examples

› **TF-IDF** › Measures **word importance in documents.**

› **LSA (Latent Semantic Analysis)** › Uses **matrix decomposition** to **find hidden relationships.**

◆ 2. Predictive Embeddings

These embeddings are **learned by training** 🧠 **Neural networks**

The model **predicts** a **target word from context** [surrounding words]



Examples

› Word2Vec

Two methods:

1) CBOW (Continuous Bag of Words)

- Predicts target word using surrounding words.
- Example: "The _ _ _ barks"
- Predict: 🐶 dog

2) Skip-Gram

- Uses current word to predict surrounding words.

› GloVe

Global Vectors for Word Representation.

Combines:

✓ Statistics

✓ Neural learning

◆ 3. Contextual Embeddings

Traditional embeddings give: ✗ One fixed meaning per word.

But contextual embeddings **change meaning depending on sentence context.**

- They **consider** the **meaning of** a **word in context** of the **entire sentence!**

Examples

› **ELMo** : **Embeddings from Language Models.**

› **BERT : Bidirectional Encoder Representations from Transformers.**

Used in:

- **ChatGPT**-like systems
- **Search engines**
- **Modern NLP**

Type	Main Idea	Examples
Count-Based	Uses co-occurrence counts	TF-IDF, LSA
Predictive	Learns via prediction	Word2Vec, GloVe
Contextual	Meaning depends on sentence	ELMo, BERT

How to Learn Word Embeddings

To learn word embeddings:

- ✓ We **design a Neural Network** (NN)
- ✓ The **network learns a fixed-length vector for every word**

The goal is: ➡ **Similar words should get similar vectors.**

What Happens During Learning?

The **model reads a huge amount of text** and **learns**:

- Which words appear together
- Which words occur in similar contexts
- Semantic relationships between words

Over time:  Meaningful embeddings are created automatically.

Steps in Embedding Learning



1. Data Preparation

- First, **collect a large text dataset (corpus)**.
- The **larger** the **corpus**: The **better** the **embeddings**.

The **model needs lots of examples to learn**:

- **Word meanings**
- **Context relationships**
- **Usage patterns**



2. Preprocessing

Before training, the **text is cleaned**.



Common Preprocessing Steps

✓ **Tokenization** > (split text into words)

Example: "I love NLP" → [I, love, NLP]

✓ **Remove noise**

- Special symbols
- Extra spaces
- Unnecessary punctuation

✓ **Lowercasing**

Example: "Dog", "dog"

treated the same.

🎯 **Goal of Preprocessing** > **Prepare clean and structured text for learning.**

3. Choose an Embedding Method

- Now select the embedding approach.
- There are 3 major types:
 - 1) **Count-based** embeddings
 - 2) **Predictive** embeddings
 - 3) **Contextual** embedding



4. Model Training

- This is the **main learning phase**.
- The **neural network** learns **vector representations**.

♦ For Count-Based Embeddings

The system:

1. **Builds** a **co-occurrence matrix**
2. Applies **dimensionality reduction** [compressed huge matrices → Smaller dense vectors]

♦ For Predictive Embeddings

- Neural networks **learn by prediction**.
- Trains models like **Word2Vec** or **GloVe** using the corpus

♦ For Contextual Embeddings

- Models like **BERT** are used.



Two Approaches

> Fine-tuning Pretrained Models

Use an **already trained BERT model** and **adapt it** to your task.

› Train From Scratch

Train the entire model on your own corpus.

Requires:

- ✗ Huge datasets
- ✗ Very high computation



5. Evaluation

After training, **test how good the embeddings are.**



Evaluation on NLP Tasks

› Word Similarity

Check **whether similar words are close.**

Example:

- king ↔ queen , cat ↔ dog

› Text Classification

Use embeddings in tasks like:

- **Spam detection**
- **Sentiment analysis**

› Language Generation

Check whether the **embeddings help generate meaningful text.**

Goal of Evaluation

Measure:

- ✓ Accuracy
 - ✓ Semantic quality
 - ✓ NLP performance
-

Step-by-Step Summary

1. Collect text data
 2. Clean & tokenize text
 3. Choose embedding method
 4. Train model
 5. Evaluate embeddings
-

Word2Vec

Word2Vec is a:

- ✓ **Neural-network-based technique**
- ✓ Used to **learn word embeddings**

It learns vector representations of words by: **Predicting words using context.**

Main idea: Words appearing in similar contexts have similar meanings.

Goal of Word2Vec

Convert words into meaningful vectors where:

- ✓ Similar words are close together
- ✓ Semantic relationships are captured

Two Versions of Word2Vec

◆ 1. CBOW (Continuous Bag of Words)

Idea: Predict missing center word using surrounding words.

◆ 2. Skip-Gram

Idea: Use one center word to predict surrounding context words.

CBOW (Continuous Bag of Words)

CBOW takes neighboring words and predicts the middle word.

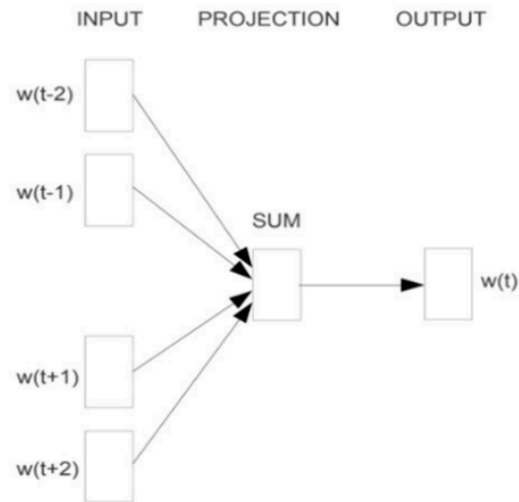
CBOW Diagram Explanation

In the diagram:

Inputs:

- $w(t-2)$
- $w(t-1)$
- $w(t+1)$
- $w(t+2)$

These are **surrounding/context words**.



What Happens?

The context word vectors are:

- ✓ **Combined (summed/averaged)**
- ✓ Passed through **projection layer**
- ✓ Used to **predict center word ($w(t)$)**

Why “Bag of Words”?

- CBOW **ignores word order**.

Example: These are treated similarly: “cat eats fish” , “fish eats cat”

Only the **presence** of words **matters**.

Important Point

Even though order is ignored:

✓ **Vector sums** still **preserve useful semantic meaning**.

> vector sum is **meaningful enough** to **deduce missing word**.

CBOW Workflow

> Takes vector embeddings of n words before target and n words after and adds them (as vectors).

Step-by-Step

1. Take **surrounding words**
 2. **Convert** to **embeddings**
 3. **Sum/average vectors**
 4. **Predict** center word
-

Skip-Gram Model

Skip-Gram does the **opposite** of CBOW.

Instead of: ✗ Context → Target

It does: ✓ **Target** → **Context**

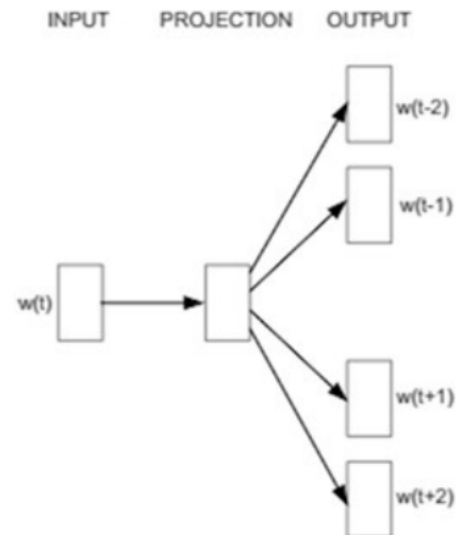
Skip-Gram Diagram Explanation

Input: **One center word ($w(t)$)**

Outputs:

- ($w(t-2)$)
- ($w(t-1)$)
- ($w(t+1)$)
- ($w(t+2)$)

The model **predicts neighboring words**.

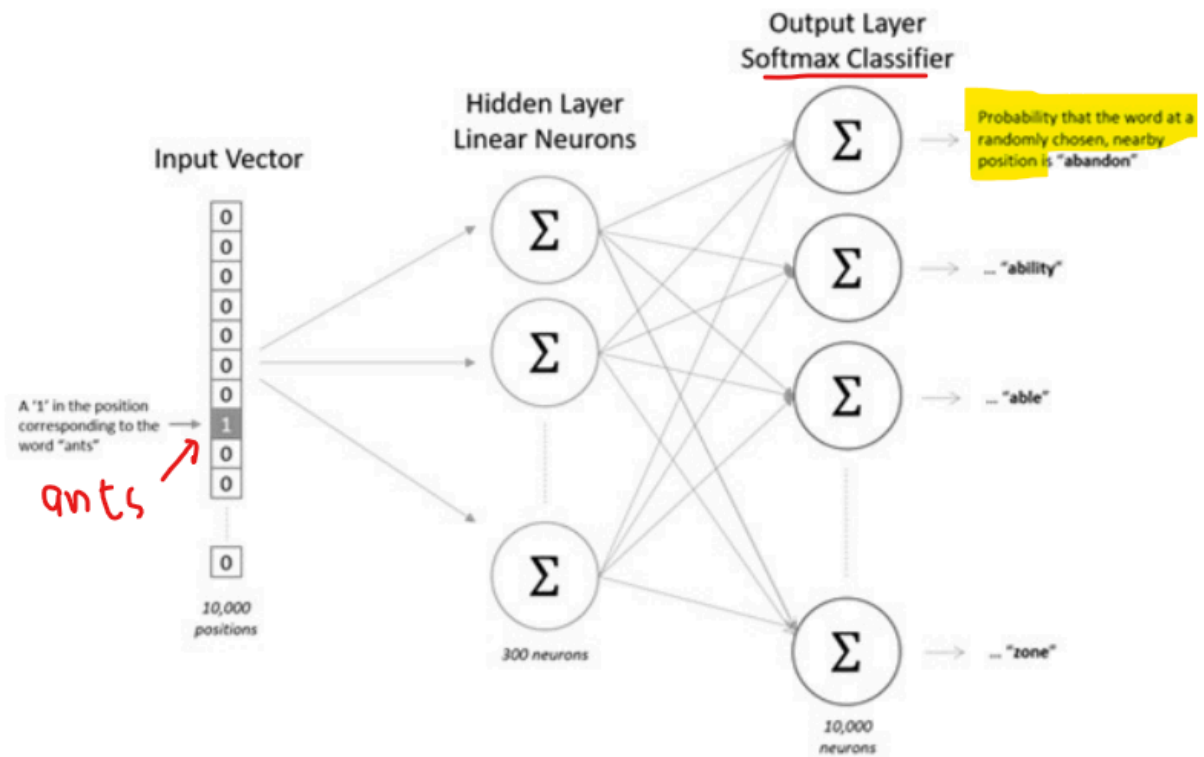


Why Skip-Gram Works Well

The model learns:

- ✓ Which words appear around a word
- ✓ Semantic relationships
- ✓ Contextual patterns

NN Diagram Explanation



◆ **Input Layer** > **one-hot vector** is given as input.

- If input word is: 🐜 "ants"
- Then: Position for "ants" = 1, All others = 0
- Vocabulary size: 10,000 words → input vector has 10,000 positions.

◆ **Hidden Layer**

- Contains **fewer neurons**: ➡ 300 neurons
- This hidden layer becomes: The **embedding layer**
- The **weights learned here are** the **word embeddings**.

◆ Output Layer

- Predicts probabilities of surrounding words.

Softmax Classifier

- The output layer uses: [Softmax Function](#)
 - It **converts scores into probabilities**.
 - **Highest probability = predicted surrounding word.**
-

CBOW vs Skip-Gram

Feature	CBOW	Skip-Gram
Predicts	Center word	Context words
Input	Surrounding words	One center word
Faster?	✓ Yes	✗ Slower
Better for rare words?	✗ Not as good	✓ Yes
Training Speed	Faster	Slower
Accuracy	Better for frequent words	Better for rare words

Which One is Better?

♦ CBOW

- ✓ Faster training
- ✓ Good for frequent/common words
- ✓ Works well on large datasets

♦ Skip-Gram

- ✓ Better semantic quality
 - ✓ Better for rare words
 - ✓ Performs better in practice
-

Positional Embeddings / encoding

Positional embedding is a technique used in **sequence models** and **transformers** to **provide information about the order** and **position of tokens in a sequence**. It helps the model **distinguish tokens based on their positions**.

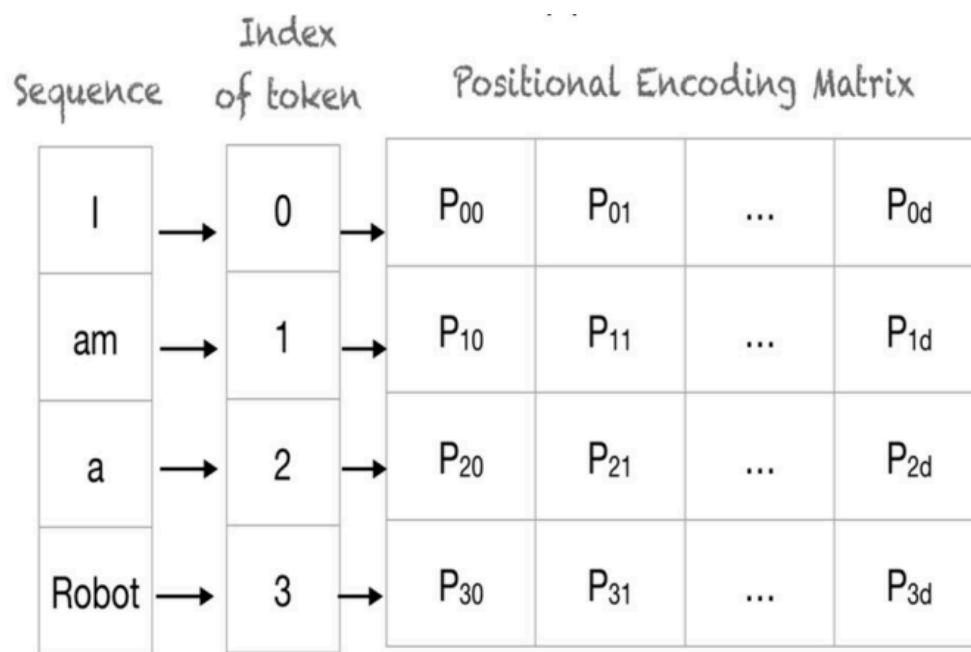
- In normal word embeddings:
 - ✓ The model knows the meaning of words
 - ✗ But it does NOT know their order in a sentence
- Positional embedding (or **positional encoding**) is a technique that adds: **Position/order information** to **each token in a sequence**

Main Purpose

- Without positional embeddings: **Transformers process all words in parallel.**
 - So they don't naturally understand sequence order.
 - Positional embeddings solve this problem.
-

Diagram Explanation

Sentence: "I am a Robot"



◆ Step 1: Token Sequence

Tokens:

- I
- am
- a
- Robot

♦ **Step 2: Assign Positions** > **Each token gets an index**

Word	Position
I	0
am	1
a	2
Robot	3

♦ **Step 3: Positional Encoding Matrix** > **Each position gets a vector**

Position	Vector
0	$P_{00}, P_{01}, \dots$
1	$P_{10}, P_{11}, \dots$
2	$P_{20}, P_{21}, \dots$

These vectors **represent positional information**.

What Happens Next?

The **positional vector** is **added to** Word embedding vector

So final representation contains:

- Meaning of word + Position of word
-

⚡ Why Positional Embeddings Matter

They help the model understand:

- ✓ Sentence structure
 - ✓ Word order
 - ✓ Grammar relationships
 - ✓ Context flow
-



BERT Embedding

BERT stands for: **Bidirectional Encoder Representations from Transformers**

> **powerful NLP model** developed by **Google**.



BERT Input Representation

BERT combines **THREE embeddings together** for each token.

- 1) **Token** embeddings
- 2) **Positional** embeddings
- 3) **Segment** embeddings

◆ 1. Token Embedding

Represents: ➡ **Meaning of the word/token**

BERT uses: **WordPiece embeddings**

WordPiece Tokenization

Words may be **broken into smaller pieces**.

Example: "playing"

may become:

- play
- ##ing

This helps handle:

- ✓ Rare words
 - ✓ Unknown words
-

◆ 2. Positional Embedding

Adds: ➡ **Position information**

So BERT knows:

- **token order**
 - **sentence structure**
-

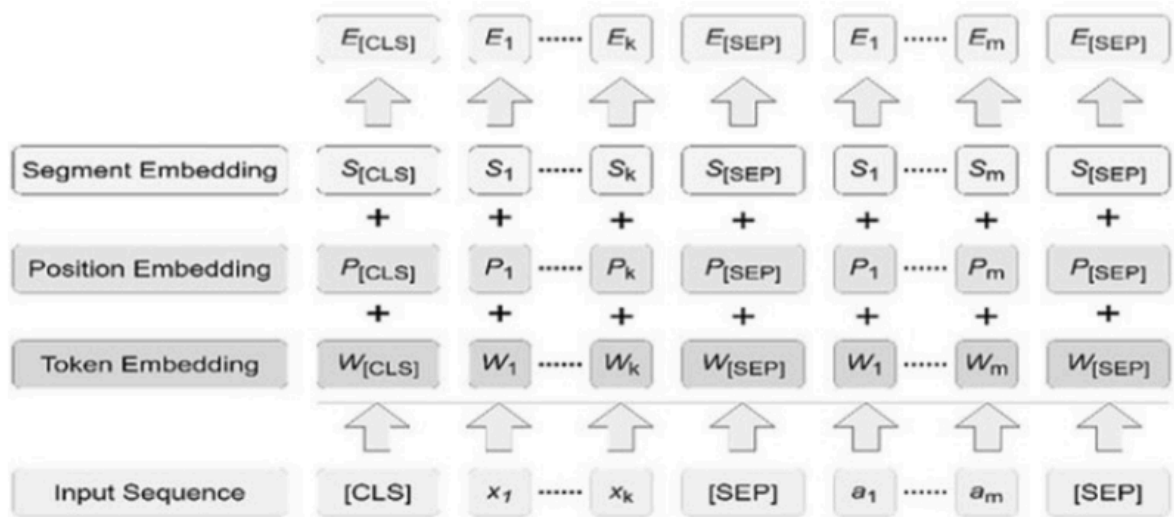
◆ 3. Segment Embedding

Used when input contains: ✓ **Two sentences** (segment pairs)

Helps **BERT distinguish**:

- **Sentence A**
- **Sentence B**

Diagram Explanation



◆ Input Sequence

Example format: $[CLS]$ Sentence A $[SEP]$ Sentence B $[SEP]$

Special Tokens

◆ $[CLS]$

- Special **classification token**.
- Used for: **sentence classification + sentiment analysis**
- The final $[CLS]$ representation summarizes the sentence.

◆ $[SEP]$

- **Separator token.**
- Used to:
 - **separate sentences**
 - **mark sentence boundaries**

◆ **Token Embeddings**

Each token gets:  **Word meaning vector**

◆ **Position Embeddings**

Each token position gets:  **Position vector**

◆ **Segment Embeddings**

Each token gets:  **Sentence identity vector**

Examples:

- **Sentence A embedding**
- **Sentence B embedding**

Final BERT Embedding

The final embedding is: **Token Embedding + Position Embedding + Segment Embedding**

$$E = W + P + S$$

Where:

- (W) = Token embedding
 - (P) = Position embedding
 - (S) = Segment embedding
-

Why BERT Embeddings Are Powerful

BERT embeddings capture:

- ✓ Word meaning
 - ✓ Word order
 - ✓ Sentence relationships
 - ✓ Bidirectional context
-

Limitations of Word Embeddings

◆ 1. Cannot Handle Same-Spelling Different Meanings Properly

- Traditional embeddings give **one vector per word**.
- **Words with multiple meanings get mixed up.**

Example:

- "rose" = flower 
- "rose" = past tense of rise 

Both may get the same embedding.

◆ 2. Needs Large Data and Memory

- **Good embeddings require huge text datasets.**
 - Large corpus increase:
 - **Memory usage**
 - **Training time**
 - **Computational cost**
-

◆ 3. Can Learn Biases

- Embeddings learn from real-world text.
 - If **data contains bias**, **embeddings learn it too.**
-



Conclusion

- Word embeddings represent words as **dense real-valued vectors**.
- They help computers understand:
 - **Meaning**
 - **Similarity**
 - **Context**
- They **improved NLP** significantly.
- Used in:
 - Chatbots
 - Search engines
 - Translation
 - Sentiment analysis

- Text classification

